

Các kỹ thuật tối ưu hóa thường gặp cho bài toán tìm kiếm trong thi tin học

Huang Xiaoyu
Trường Trung học số 1 Trùng Khánh

Nguồn gốc: Common optimization techniques for search problems in informatics contests

IOI China candidate team papers / 2006 / Huang Xiaoyu.doc

Abstract

Bài viết kết hợp các ví dụ để phân tích và tổng kết những phương pháp suy nghĩ, phương pháp giải bài thường dùng khi xử lý bài toán tìm kiếm trong thi tin học. Từ hai phía tìm kiếm theo chiều sâu và tìm kiếm theo chiều rộng, bài viết thảo luận các kỹ thuật thích hợp để nâng cao hiệu suất chương trình.

Từ khóa: tin học; thứ tự tìm kiếm; đối tượng tìm kiếm; bảng băm; cắt tỉa.

1 Mở đầu

Trong thi tin học, khi giải bài toán tìm kiếm, ta thường dùng hai phương pháp: tìm kiếm theo chiều sâu và tìm kiếm theo chiều rộng.

2 Kỹ thuật tối ưu hóa tìm kiếm theo chiều sâu

Khi làm bài, ta thường gặp loại bài như sau: cho các điều kiện ràng buộc, yêu cầu tìm một phương án thỏa các điều kiện ấy. Ta gọi lớp bài này là bài toán **thỏa ràng buộc**. Với bài toán thỏa ràng buộc, thông thường có thể bắt đầu từ thứ tự tìm kiếm và đối tượng tìm kiếm, rồi nhờ đó nâng cao hiệu suất chương trình.

2.1 Thứ tự và đối tượng tìm kiếm

Khi giải bài toán thỏa ràng buộc, điều kiện ràng buộc mà đề bài đưa ra càng mạnh thì càng có lợi cho việc cắt tỉa trong tìm kiếm. Sở dĩ hiệu quả của tìm kiếm theo chiều sâu thường tốt hơn rất nhiều so với vét cạn là vì trong quá trình tìm kiếm, nó tận dụng tốt các điều kiện ràng buộc của bài toán để cắt tỉa, từ đó nâng cao hiệu suất chương trình.

Hiển nhiên, trên cùng một cây tìm kiếm, nếu có thể dùng điều kiện ràng buộc để cắt tỉa càng gần gốc thì hiệu quả càng tốt. Làm thế nào để tận dụng tối đa các điều kiện ràng buộc của bài toán làm căn cứ cắt tỉa là một điểm rất quan trọng để nâng cao hiệu quả tìm kiếm theo chiều sâu. Thứ tự tìm kiếm và đối tượng tìm kiếm khác nhau sẽ ảnh hưởng trực tiếp tới cách ta sử dụng các điều kiện ràng buộc ấy.

Dưới đây ta xét ảnh hưởng của các phương pháp khác nhau từ hai phía: lựa chọn thứ tự tìm kiếm và lựa chọn đối tượng tìm kiếm.

2.2 Lựa chọn thứ tự tìm kiếm

Trước hết xét một bài tương đối đơn giản: **ZJU 1937**. Cho một dãy

$$a_0, a_1, \dots, a_m$$

thỏa

$$a_0 = 1, \quad a_m = n, \quad a_0 < a_1 < a_2 < \dots < a_{m-1} < a_m.$$

Với mỗi k , $1 \leq k \leq m$, có

$$a_k = a_i + a_j \quad (0 \leq i, j \leq k-1),$$

trong đó i và j có thể bằng nhau. Cho trước n , yêu cầu tìm giá trị nhỏ nhất của m (không cần in ra) và một dãy tương ứng; nếu có nhiều dãy thỏa mãn thì chỉ cần in ra một dãy bất kỳ.

Vì $a_k = a_i + a_j$ với $0 \leq i, j < k$, trong quá trình tìm kiếm ta có thể thử tính theo thứ tự từ nhỏ đến lớn từng phần tử của dãy. Khi tìm kiếm thông thường, ta quen xét lần lượt giá trị của mỗi số từ nhỏ đến lớn. Nhưng trong bài này, nếu lập trình và chạy theo thứ tự ấy thì hiệu quả rất không tốt:

N	10	20	30	40	50	60	70	80	90	100	200	300	400	500
Thời gian	0.03	0.01	0.03	0.05	0.20	0.34	1.80	1.80	8.91	10.1	quá lâu	quá lâu	quá lâu	quá lâu

Vì đề bài yêu cầu m nhỏ nhất, tức là cần đạt được số n càng nhanh càng tốt, nên mỗi số được dựng ra nên càng lớn càng tốt. Dựa trên đặc tính này, ta đổi thứ tự tìm kiếm thành xét mỗi số từ lớn đến nhỏ. Hiệu quả của chương trình mới như sau:

N	10	20	30	40	50	60	70	80	90	100	200	300	400	500
Thời gian	0.01	0.01	0.01	0.01	0.01	0.01	0.03	0.01	0.03	0.03	0.13	1.48	1.5	22.88

Rõ ràng chương trình dùng thứ tự tìm kiếm sau hiệu quả hơn hẳn chương trình dùng thứ tự đầu tiên.

Dĩ nhiên bài này vẫn còn rất nhiều chỗ có thể tối ưu, nhưng tư tưởng về thứ tự tìm kiếm được dùng rất rộng rãi trong các bài tìm kiếm. Có thể mọi người cho rằng cách chỉ dùng thứ tự tìm kiếm để tối ưu chương trình là khá bình thường, nhưng phương pháp có vẻ đơn giản này xuất hiện không ít trong các kỳ thi. Chẳng hạn bài **Block** của IOI 2000: chỉ cần xét các khối từ lớn đến nhỏ, sau khi xoay và lật thì lần lượt đặt vào để tìm kiếm, dữ liệu trong kỳ thi đã có thể đạt điểm tối đa. Một ví dụ gần đây là bài **Trí tuệ châu** của NOI 2005: cũng chỉ cần tìm kiếm các viên từ lớn đến nhỏ, không thêm bất kỳ cắt tỉa nào khác, vẫn có thể đạt 90 điểm trong kỳ thi.

Có thể thấy, chọn thứ tự tìm kiếm thích hợp là một trong các kỹ thuật thiết kế chương trình hiệu quả nhất để nâng cao hiệu suất. Dùng thứ tự tìm kiếm tốt để tối ưu bài toán tìm kiếm là một thuật toán có tỷ lệ lợi ích trên chi phí rất cao.

2.3 Lựa chọn đối tượng tìm kiếm

Tiếp theo xét bài **USACO Weight**. Ta biết các tổng của $1, 2, 3, \dots, n$ phần tử đầu của dãy gốc $a_1, a_2, \dots, a_n$, đồng thời biết các tổng của $1, 2, 3, \dots, n$ phần tử cuối. Tuy nhiên tất cả dữ liệu này đã bị xáo trộn thứ tự. Ngoài ra biết rằng các số trong dãy thuộc tập S . Hãy khôi phục dãy gốc. Nếu có nhiều dãy khả dĩ, cần tìm dãy có phần bên trái nhỏ nhất.

Trong đó $n \leq 1000$, $S \in \{1, \dots, 500\}$. Ví dụ, nếu dãy gốc là

$$1 \quad 1 \quad 5 \quad 2 \quad 5,$$

$S = \{1, 2, 4, 5\}$, thì các giá trị đã biết là

$$(1, 2, 7, 9, 14, 5, 7, 12, 13, 14).$$

Cụ thể:

$$\begin{array}{ll} 1 = 1 & 5 = 5, \\ 2 = 1 + 1 & 7 = 2 + 5, \\ 7 = 1 + 1 + 5 & 12 = 5 + 2 + 5, \\ 9 = 1 + 1 + 5 + 2 & 13 = 1 + 5 + 2 + 5, \\ 14 = 1 + 1 + 5 + 2 + 5 & 14 = 1 + 1 + 5 + 2 + 5. \end{array}$$

Vì $S \in \{1, \dots, 500\}$, trong trường hợp xấu nhất mỗi số có thể nhận 500 giá trị. Rất khó tìm được một phương pháp toán học tốt để giải quyết, còn dùng tìm kiếm lại là một cách khá thích hợp. Nếu lần lượt từ trái sang phải tìm giá trị có thể của từng số trong dãy gốc rồi so sánh với các giá trị đã biết, ta thu được phương pháp tìm kiếm đơn giản nhất, gọi là phương pháp A.

Nhưng trong trường hợp xấu nhất, phương pháp A mở rộng 500^{1000} nút, tốc độ quá chậm. Trong thuật toán này, mỗi số của dãy đều được tìm kiếm 500 lần, dẫn tới lượng tìm kiếm rất lớn. Giảm lượng tìm kiếm một cách hiệu quả là chìa khóa để nâng cao hiệu quả thuật toán của bài này. Phương pháp dùng thứ tự tìm kiếm đã nói ở trên lại không áp dụng được ở đây, vì đề bài quy định dãy cần có phần bên trái nhỏ nhất. Ta hãy đổi góc nhìn.

Trong phương pháp tìm kiếm B, quay lại xét các điều kiện ràng buộc mà đề bài cung cấp. Dùng S_i biểu thị tổng i phần tử đầu, và T_i biểu thị tổng i phần tử cuối. Theo đề bài, dữ liệu nhận được chính là

$$S_1, S_2, S_3, \dots, S_n \quad \text{và} \quad T_1, T_2, T_3, \dots, T_n.$$

Mọi hiệu $S_{i+1} - S_i$ và $T_{i+1} - T_i$ đều thuộc tập S . Một điều kiện ràng buộc khá dễ thấy khác là với mọi i , ta có

$$S_n = T_n = S_i + T_{n-i}.$$

Tương tự, trong quá trình tìm kiếm, tận dụng tối đa các điều kiện này là chìa khóa để nâng cao hiệu quả chương trình.

Khi lấy tùy ý hai số trong dữ liệu đã biết, chỉ có hai trường hợp:

1. Hai số cùng thuộc dãy S_i , hoặc cùng thuộc dãy T_i .
2. Hai số lần lượt thuộc T_i và S_i .

Khi hai số cùng thuộc S_i hoặc cùng thuộc T_i , hiệu của hai số chính là đoạn $S_j - S_i$ trong hình ở bản gốc; khi $j = i + 1$, $S_j - S_i$ tất yếu thuộc tập S mà đề bài cho. Vì vậy, mỗi khi thu được một số S_i hoặc T_i , nếu đã biết S_{i-1} hoặc T_{i-1} , ta có thể phán đoán S_i hoặc T_i hiện tại có hợp lệ hay không. Do đó trong tìm kiếm, nếu cố gắng dùng S_{i-1} và T_{i-1} để suy ra các khả năng của S_i và T_i , ta có thể tận dụng điều kiện ràng buộc của bài toán nhiều nhất có thể.

Vì các điều kiện ràng buộc của đề bài tập trung vào S_i và T_i , ta thay đổi đối tượng tìm kiếm: không tìm giá trị của từng số trong dãy gốc nữa, mà tìm vị trí của các số đã cho khi chúng xuất hiện trong S_i hoặc T_i . Hơn nữa, quan hệ ràng buộc giữa S_{i+1} và S_i gợi ý rằng trong tìm kiếm nên xét theo thứ tự tăng hoặc giảm của chỉ số i trong dãy S_i .

Ví dụ, với nhóm dữ liệu 1, 1, 5, 2, 5, các giá trị sinh ra là

$$1, 2, 7, 9, 14, 5, 7, 12, 13, 14.$$

Sau khi sắp xếp:

$$1, 2, 5, 7, 7, 9, 12, 13, 14, 14.$$

Hai số lớn nhất là tổng của toàn bộ dãy, nên trong tìm kiếm không cần xét chúng. Bỏ 14 đi, còn

$$1, 2, 5, 7, 7, 9, 12, 13.$$

Quan sát thấy số nhỏ nhất của dãy là 1, chỉ có thể xuất hiện ở đầu hoặc cuối dãy cần tìm. Giả sử đã biết vị trí của 1, sau khi bỏ nó đi, xét số nhỏ nhất còn lại là 2; rõ ràng 2 cũng chỉ có thể nhận được bằng cách thêm một số vào đầu hoặc cuối trạng thái hiện tại. Như vậy, mỗi khi tìm một số, ta chỉ đặt nó vào đầu hoặc cuối, tức là đặt vào dãy S_i hoặc dãy T_i .

Suy rộng ra, ta tìm kiếm các số đã sắp xếp theo thứ tự tăng dần, phán đoán mỗi số xuất hiện ở đầu hay ở cuối dãy gốc. Lúc này ta tìm kiếm từ hai đầu của dãy gốc vào giữa, thay vì tìm từ một đầu sang đầu kia như trước. Từ phân tích ở trên, mỗi số chỉ có thể thuộc S_i hoặc T_i . Khi đã tìm được $S_1, S_2, \dots, S_i$ và $T_1, T_2, \dots, T_j$ của dãy gốc, với số K đang xét chỉ có hai khả năng tồn tại: S_{i+1} và T_{j+1} . Ta lần lượt tìm kiếm hai khả năng ấy, tức là phán đoán $K - S_i$ và $K - T_j$ có thuộc tập S đã biết hay không. Đồng thời, mỗi khi tìm được một số K , ta xóa $S_n - K$ khỏi dãy đã sắp xếp. Như vậy, khi $K - S_i$ (hoặc $K - T_j$) không thuộc tập S , hoặc $S_n - K$ không tồn tại trong dãy đã sắp xếp, ta quay lui.

Thuật toán này trong trường hợp xấu nhất mở rộng 2^{1000} nút (trong thực tế nhỏ hơn rất nhiều). Do trong quá trình tìm kiếm nó tận dụng đầy đủ các điều kiện ràng buộc của bài toán, kết quả chạy chương trình tốt hơn nhiều.

Trong bài này, phương pháp tìm kiếm ban đầu có lượng tìm kiếm khổng lồ. Bằng cách phân tích và chọn đối tượng tìm kiếm thích hợp, ta vừa giảm lượng tìm kiếm, vừa tận dụng đầy đủ các điều kiện ràng buộc của đề bài, khiến chúng trở thành cốt lõi có lợi cho chương trình, nhờ đó bài toán được giải quyết tốt.

3 Kỹ thuật tối ưu hóa tìm kiếm theo chiều rộng

So với tìm kiếm theo chiều sâu, một loại bài khác là: cho trạng thái bắt đầu, trạng thái đích và quy tắc chuyển trạng thái, yêu cầu tìm một đường đi hoặc một phương pháp để đạt tới trạng thái đích. Ta gọi lớp bài này là bài toán **tìm đường** (chẳng hạn bài đi mê cung). Cách hiệu quả nhất để giải những bài này là chọn phương pháp xây dựng bảng băm thích hợp.

Các phương pháp xây dựng bảng băm thường gặp gồm:

- Nén trạng thái: dùng biểu diễn nhị phân để ghi trạng thái.
- Lấy dư trực tiếp: chọn một số nguyên tố M làm số chia.
- Bình phương lấy giữa: tính bình phương khóa, rồi lấy r chữ số ở giữa để tạo một bảng kích thước 2^r .
- Gấp đoạn: cộng các mã ASCII của toàn bộ ký tự.

Trong bài toán tìm đường, ta thường gặp vấn đề đi vòng trở lại, nên trong quá trình tìm kiếm bắt buộc phải làm một việc: kiểm tra trùng lặp. Kiểm tra trùng lặp là then chốt quyết định hiệu quả chương trình, còn cách xây dựng một bảng băm tốt lại quyết định việc ấy. Một hàm băm tốt có thể nâng cao đáng kể hiệu quả thời gian và không gian tổng thể của chương trình.

3.1 Ví dụ ZJU 1301

Ông Black mới mua một biệt thự, nhưng hệ thống dây điện bên trong rất rối (công tắc trong mỗi phòng có thể điều khiển phòng khác, số phòng không quá 10). Một tối nọ, khi về nhà, ông phát hiện mọi đèn đều tắt, trừ phòng xuất phát của ông, và ông muốn về phòng ngủ để nghỉ. Không may là ông rất sợ bóng tối, nên sẽ không bước vào bất kỳ phòng nào đang tắt đèn. Hãy tìm giúp ông một đường đi để vừa về được phòng ngủ, vừa tắt hết các đèn ngoài phòng ngủ. Nếu có nhiều đường đi, hãy in đường ngắn nhất.

Đây là một bài tìm kiếm khá đơn giản. Đề yêu cầu một đường đi, nên ta dùng tìm kiếm theo chiều rộng để giải. Điều mà tìm kiếm theo chiều rộng không thể tránh được là các trạng thái lặp. Dùng vòng

lập để kiểm tra trùng là không đáng; khi số trạng thái lớn, phương pháp vòng lặp thậm chí kém hiệu quả hơn tìm kiếm theo chiều sâu rất nhiều. Khó khăn của bài nằm ở việc xây dựng bảng băm. Qua phân tích, các yếu tố ảnh hưởng đến trạng thái là việc đèn trong mỗi phòng bật hay tắt, và phòng hiện tại của ông Black. Vì đèn chỉ có hai trạng thái bật và tắt, ta xét dùng nhị phân để lưu trạng thái, tức nén trạng thái quen thuộc.

Biểu diễn trạng thái đèn trong mỗi phòng bằng 0 và 1, rồi ghép trạng thái của 10 phòng lại, ta được dạng như

$$1000100101.$$

Chuyển nó sang thập phân:

$$(1000100101)_2 = (549)_{10}.$$

Như vậy có thể biểu diễn duy nhất một trạng thái bật tắt đèn. Thêm một số để ghi phòng hiện tại của ông Black, ta xây dựng xong bảng băm cần thiết. Tổng số trạng thái là

$$2^{10} \cdot 10 = 10240.$$

Đồng thời, trong tìm kiếm có thể dùng phép toán bit để phán đoán trạng thái của một phòng, khiến việc điền và tra bảng băm trở nên đơn giản. Ví dụ, giả sử trạng thái hiện tại là K , cần phán đoán trạng thái của phòng thứ i . Chỉ cần xem $(2^{i-1} \text{ and } K)$ có bằng 0 hay không. Đến đây bài toán đã được giải quyết.

3.2 Ví dụ PKU 1729

Trên một bản đồ $N \times N$ ($N \leq 30$) có hai người A và B . Một số ô trên bản đồ là đất trống, một số ô là chướng ngại không thể đi qua. Tại mỗi thời điểm, A và B đều phải di chuyển theo một trong bốn hướng N, E, W, S. Hai người rất ghét nhau, nên khi di chuyển họ muốn cách xa nhau nhất có thể. Biết điểm xuất phát và điểm đích của mỗi người, hãy tìm một đường đi để A và B cùng tới đích, sao cho khoảng cách nhỏ nhất giữa họ trên đường đi là lớn nhất; trên cơ sở đó, hãy để họ tới đích sớm nhất. Bản gốc có kèm hình minh họa cho một trường hợp $N = 10$.

Đây là bài tìm đường, nên rõ ràng là một bài tìm kiếm theo chiều rộng. Điều kiện của bài rất nhiều: trước hết A và B đều phải tới đích, sau đó khoảng cách giữa A và B trên đường đi phải càng xa càng tốt, đồng thời họ phải tới đích càng nhanh càng tốt.

Trước hết ta thử dùng bảng băm để lưu toàn bộ thông tin rồi tìm kiếm. Ban đầu có thể nghĩ tới hai cách xây dựng bảng băm:

- Dùng Hash[x1, y1, x2, y2, t] để biểu thị: sau t thời điểm, A tới tọa độ (x_1, y_1) , B tới (x_2, y_2) , và khoảng cách nhỏ nhất giữa họ trên đường đi.
- Dùng Hash[x1, y1, x2, y2, k] để biểu thị: khi A tới (x_1, y_1) , B tới (x_2, y_2) , và khoảng cách nhỏ nhất giữa họ trên đường đi là k , thời gian ít nhất cần dùng.

Điểm chung của hai cách này là bảng băm phải lưu một lượng thông tin rất lớn, và hiện thực lập trình cũng tương đối khó. Vì quá nhiều điều kiện bị dồn vào bảng băm, ta thử bỏ một điều kiện khỏi bảng băm và dùng phương pháp khác để chia sẻ gánh nặng.

Giả sử đã biết khoảng cách nhỏ nhất giữa hai người trên đường đi, phần còn lại sẽ đơn giản hơn nhiều. Nhưng làm sao biết được khoảng cách nhỏ nhất ấy? Ta nghĩ tới tìm kiếm nhị phân. Chênh lệch khoảng cách giữa hai người trên bản đồ nhiều nhất chỉ có thể có $30^4 = 810000$ khả năng. Nếu dùng nhị phân thì cần nhiều nhất $\log_2 810000 < 20$ lần, rồi dùng tìm kiếm theo chiều rộng để giải phần còn lại. Trong trường hợp xấu nhất, số nút mở rộng là

$$20 \cdot 30^4 = 16200000,$$

hoàn toàn có thể cho kết quả trong thời gian quy định.

Xét rằng trong quá trình A và B di chuyển, cùng một trạng thái (x_1, y_1, x_2, y_2) không thể xuất hiện hai lần, có thể trực tiếp dùng bảng băm để lưu tình huống của trạng thái (x_1, y_1, x_2, y_2) . Khi đó bảng băm nên lưu: A tới tọa độ (x_1, y_1) , B tới (x_2, y_2) , khoảng cách nhỏ nhất giữa họ trên đường đi, và trên cơ sở ấy là thời gian ngắn nhất.

3.3 Tiểu kết

Bảng băm là một trong các phương pháp tối ưu hóa tìm kiếm theo chiều rộng rất quan trọng. Nó có thể nâng hiệu quả của thuật toán tìm kiếm từ mức mũ lớn lên mức mũ nhỏ, mức đa thức, thậm chí mức hằng số.

4 Công cụ mạnh cho cả hai loại tìm kiếm: cắt tỉa

4.1 USACO Cryptcowgraphy

Các con bò của nông dân Brown và John dự định phối hợp trốn khỏi các nông trại của mình. Chúng thiết kế một phương pháp mã hóa để bảo vệ thông tin liên lạc khỏi người khác.

Nếu một con bò muốn mã hóa một thông điệp, chẳng hạn

International Olympiad in Informatics,

nó sẽ ngẫu nhiên chèn ba chữ cái C, O, W vào thông điệp, trong đó C đứng trước O, và O đứng trước W. Sau đó nó hoán đổi vị trí của đoạn văn bản giữa C và O với đoạn văn bản giữa O và W. Hai ví dụ là:

```
International Olympiad in Informatics -> CnOIWternational Olympiad in Informatics
International Olympiad in Informatics -> International Cin InformaticsOOlympiad W
```

Để việc giải mã phức tạp hơn, các con bò có thể áp dụng phương pháp mã hóa này nhiều lần trên một thông điệp, tức là tiếp tục mã hóa kết quả của lần mã hóa trước. Một đêm nọ, đàn bò của John nhận được một thông điệp đã qua nhiều lần mã hóa. Hãy viết chương trình phán đoán liệu nó có thể thu được bằng cách mã hóa (hoặc không mã hóa) thông điệp sau hay không:

Begin the Escape execution at the Break of Dawn

Dựa trên quy tắc mã hóa, ta rất dễ nghĩ ra một phương pháp dfs cực kỳ đơn giản; tất nhiên phương pháp ấy rõ ràng sẽ quá thời gian. Phân tích bài toán cho thấy đề yêu cầu một cách mã hóa để thu được thông tin, tức là tìm một đường mã hóa. Vì vậy ta buộc phải dùng thuật toán tìm kiếm theo chiều rộng.

Với cách xây dựng bảng băm, có thể dùng hàm ELFhash hoặc SDBMhash (xem bài của Li Yuxiu năm 2005). Ở đây bỏ qua phần này.

Đề quy định thông điệp mã hóa không quá 75 ký tự, còn thông điệp gốc có tất cả 47 ký tự. Như vậy, theo cách mã hóa hợp lệ, nhiều nhất có thể chèn 9 nhóm ký tự C, O, W. Nếu dùng thuật toán ban đầu để tìm kiếm, nhiều nhất sẽ mở rộng $(9!)^3$ nút, vượt xa giới hạn thời gian, nên cắt tỉa là bắt buộc.

Đề bài đã cho thông điệp gốc, do đó việc tận dụng tốt thông tin đã biết để cắt tỉa sẽ ảnh hưởng trực tiếp tới hiệu quả chương trình.

1. Mỗi lần mã hóa mật văn đều thêm ba chữ cái C, O, W, nên độ dài mật văn nhập vào phải có dạng $47 + 3n$. Điều này có thể nhanh chóng loại một số dữ liệu.

2. Số lần xuất hiện của mỗi ký tự trong mật văn, ngoài C, O, W, chắc chắn là cố định. Số lần xuất hiện của C, O, W cũng chắc chắn tăng thêm $(n - 47)/3$ so với thông điệp gốc, trong đó n là độ dài thông điệp nhập vào.
3. Các chữ C, O, W xuất hiện trong nguyên văn đều là chữ thường. Vì vậy trong thông điệp đã mã hóa, các đoạn ký tự liên tiếp giữa C và O, cũng như giữa O và W, bắt buộc phải là các đoạn con của nguyên văn.
4. Đoạn giữa ký tự đầu tiên và ký tự mã hóa C phải thuộc nguyên văn. Tương tự, đoạn giữa ký tự cuối cùng và ký tự mã hóa W cuối cùng cũng phải thuộc nguyên văn.

Trong hiện thực chương trình, để tìm kiếm nhanh hơn, ta lưu các ký tự mã hóa trong chuỗi theo chỉ số. Khi đó thời gian mở rộng mỗi nút giảm mạnh, trường hợp xấu nhất là 9^3 . Nếu không làm vậy, mỗi lần mở rộng nút sẽ tốn $(\text{length}(s))^3$.

Sau khi dùng các tối ưu này, hiệu quả chương trình sẽ tăng lên rất nhiều.

4.2 Tiểu kết

Cắt tĩa là trọng điểm không thể thiếu trong mọi bài tìm kiếm. Tận dụng đầy đủ đặc tính của bài toán để cắt tĩa có thể nâng cao hiệu quả chương trình ở mức rất lớn.

Tài liệu tham khảo

1. Liu Rujia, Huang Liang. *Nghệ thuật thuật toán và thi tin học* (2004).
2. AngleForYou. *Các phương pháp tối ưu hóa chung cho thuật toán tìm kiếm*.
3. USACO. *Báo cáo lời giải Cryptcowgraphy*.